

A. Database collection: voti.json

1. Trovare per ogni studente il numero di corsi che ha seguito

```
db.grades.aggregate([
  {
    $group: {
      _id: "$codiceStudente",
      numeroCorsi: { $sum: 1 }
    }
  },
  {
    $sort: { _id: 1 }
  }
])
```

2. Per ogni coppia studente-corso trovare la media dei voti

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $group: {
      _id: { codiceStudente: "$codiceStudente",
              codiceCorso: "$codiceCorso" },
      media: {
        $avg: "$valutazioni.voto"
      }
    }
  }
])
```

3. Per ogni coppia studente-corso trovare la media dei voti dei soli "homework"

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $match: {
      "valutazioni.tipo": "homework"
    }
  },
  {
    $group: {
      _id: { codiceStudente: "$codiceStudente",
              codiceCorso: "$codiceCorso" },
      media: {
        $avg: "$valutazioni.voto"
      }
    }
  }
])
```

4. Trovare, per ogni corso, il numero di studenti che lo hanno seguito, in ordine di codice corso

```
db.grades.aggregate([
  {
    $group: {
      _id: "$codiceCorso",
      numeroCorsi: { $sum: 1 }
    }
  },
  {
    $sort: { _id: 1 }
  }
])
```

5. Per ogni studente trovare la media dei voti dei soli "homework" (calcolata tra tutti i corsi seguiti)

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $match: {
      "valutazioni.tipo": "homework"
    }
  },
  {
    $group: {
      _id: {codiceStudente: "$codiceStudente"},
      media: {
        $avg: "$valutazioni.voto"
      }
    }
  }
])
```

6. Per ogni studente calcolare la media dei voti (la media delle medie nei vari corsi)

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $group: {
      _id: {codiceStudente: "$codiceStudente",
            codiceCorso:"$codiceCorso"},
      media: {
        $avg: "$valutazioni.voto"
      }
    }
  },
  {
    $group: {
      _id: "$_id.codiceStudente",
      media: {
        $avg: "$media"
      }
    }
  }
])
```

7. Per ogni corso trovare la media dei voti degli studenti (la media della media degli studenti)

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $group: {
      _id: {codiceStudente: "$codiceStudente",
            codiceCorso:"$codiceCorso"},
      media: {
        $avg: "$valutazioni.voto"
      }
    }
  },
  {
    $group: {
      _id: "$_id.codiceCorso",
      media: {
        $avg: "$media"
      }
    }
  }
])
```

8. Trovare lo studente con il voto più basso

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $sort: {"valutazioni.voto": 1}
  },
  {
    $limit: 1
  }
])
```

9. Per ogni corso trovare lo studente con la media più alta e quello con la media più bassa

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $group: {
      _id: {codiceStudente: "$codiceStudente",
            codiceCorso:"$codiceCorso"},
      media: {
        $avg: "$valutazioni.voto"
      }
    }
  },
  {
    $sort:{media: -1}
  },
  {
    $group: {
      _id: "$_id.codiceCorso",
      studente_media_alta: {
        $first: "$_id.codiceStudente"
      },
      studente_media_bassa: {
        $last: "$_id.codiceStudente"
      }
    }
  }
])
```

10. Trovare il numero totale "homework" assegnati

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni"
  },
  {
    $match: { "valutazioni.tipo": "homework" }
  },
  {
    $count: "totale_homework"
  }
])
```

11. Trovare la tipologia di valutazione più comune

```
db.grades.aggregate([
  {
    $unwind: "$valutazioni",
  },
  {
    $group: {
      _id: "$valutazioni.tipo",
      numero: {
        $sum: 1,
      },
    },
  },
  {
    $sort:
      /**
       * Provide any number of field/order pairs.
       */
      {
        numero: -1,
      },
  },
  {
    $limit:
      /**
       * Provide the number of documents to limit.
       */
      1,
  },
  {
    $replaceRoot:
      /**
       * replacementDocument: A document or string.
       */
      {
        newRoot: {
          tipo: "$_id",
          numero: "$numero",
        },
      },
  },
],
```

B: Database collection: books.json

12. Trovare i libri che sono stati pubblicati tra il 2009 e 2011 (status "PUBLISH")

```
db.books.aggregate([
  {
    $project: {
      title: 1,
      status: 1,
      annoPub: {$year: "$publishedDate"}
    }
  },
  {
    $match :{
      annoPub: {$gt: 2009, $lt:2011},
      status: "PUBLISH"
    }
  }
])
```

13. Trovare quanti libri sono stati pubblicati per ogni autore con il nome che inizia con "C"

```
db.books.aggregate([
  {
    $unwind: "$authors"
  },
  {
    $match: { authors: { $regex: /^c.*/i }}
  },
  {
    $group: {
      _id: "$authors",
      libri_pub:{$sum: 1}
    }
  }
])
```

14. Trovare il libro con il maggior numero di autori

```
db.books.aggregate([
  {
    $project:{
      title: 1,
      n_autori: {$size: "$authors"}
    }
  },
  {
    $sort: {
      n_autori: -1
    }
  },
  {
    $limit: 1
  }
])
```

15. Per l'autore "Robi Sen" trovare il totale delle pagine che ha scritto

```
db.books.aggregate([
  {
    $unwind: "$authors"
  },
  {
    $match: {
      authors: "Robi Sen"
    }
  },
  {
    $group: {
      _id: "$authors",
      tot_pagine: {$sum : "$pageCount"}
    }
  }
])
```

16. Trovare l'anno nel quale sono stati pubblicati il maggior numero di libri

```
db.books.aggregate([
  {
    $project: {
      title: 1,
      status: 1,
      annoPub: {$year: "$publishedDate"}
    }
  },
  {
    $match: { status: "PUBLISH" }
  },
  {
    $group: {
      _id: "$annoPub",
      count: {$count: {}}
    }
  },
  {
    $sort: {count: -1}
  },
  {
    $limit: 1
  }
])
```

17. Trovare il libro con il titolo più lungo

```
db.books.aggregate([
  {
    $project: {
      title: 1,
      len: {$strLenCP: "$title"}
    }
  },
  {
    $sort: { len: -1 }
  },
  {
    $limit: 1
  }
])
```

18. Per ogni autore trovare il totale delle pagine che ha "scritto" (somma delle pagine dei libri che ha pubblicato)

```
db.books.aggregate([
  {
    $unwind: "$authors"
  },
  {
    $group: {
      _id: "$authors",
      somma_pag: {$sum: "$pageCount"}
    }
  }
])
```

19. Trovare le 3 categorie più numerose.

```
db.books.aggregate([
  {
    $unwind: "$categories"
  },
  {
    $group: {
      _id: "$categories",
      count: {$sum: 1}
    }
  },
  {
    $sort: {count: -1}
  },
  {
    $limit: 1
  }
])
```

20. Per ogni mese trovare la quantità di pubblicazioni

```
db.books.aggregate([
  {
    $project: {
      month: {$month: "$publishedDate"}
    }
  },
  {
    $group: {
      _id: "$month",
      freq: {$sum: 1}
    }
  }
])
```

Database: sales.json

21. Trovare il numero di vendite online a London.

```
db.sales.aggregate([
  {
    $match: {
      "storeLocation": "London",
      "purchaseMethod": "Online"
    }
  }, {
    $count: "numeroVendite"
  }
])
```

22. Trovare il totale di vendite online nell'anno 2015

```
db.sales.aggregate(
  [
    {
      $project: {
        year: { $year: "$saleDate" },
      }
    },
    {
      $match: {
        "year": 2015
      }
    }, {
      $count: "totaleVendite"
    }
  ]
)
```

23. Trovare i 5 cliente più giovani di sesso maschile, proiettare solo l'email e l'età.

```
db.sales.aggregate([
  {
    $match: {
      "customer.gender": "M",
    }
  },
  {
    $project: {
      _id: 0,
      "customer.email": 1,
      "customer.age": 1
    }
  },
  {
    $sort: {
      "customer.age": 1
    }
  },
  {
    $limit: 5
  }
])
```

24. Trovare per ogni metodo di acquisto, la media di soddisfazione dei customer, il numero di customer e l'età media dei customer.

```
db.verifica.aggregate([
  {
    $group: {
      _id: "$purchaseMethod",
      media: { $avg: "$customer.statisfaction" },
      somma: { $sum: 1 },
      mediaEtà: { $avg: "$customer.age" }
    }
  }
])
```

25. Trovare per ogni metodo di acquisto, il numero di acquisti con e senza coupon.

```
db.sales.aggregate([
  {
    $group: {
      _id: { "metodoAcquisto": "$purchaseMethod",
            "couponUsed": "$couponUsed" },
      tot: { $sum: 1 }
    }
  }
])
```

26. Trovare la città dove sono stati usati il maggior numero di coupon.

```
db.sales.aggregate([
  {
    $match: {
      "couponUsed": true
    }
  }, {
    $group: {
      _id: "$storeLocation",
      totCoupon: { $sum: 1 }
    }
  }, {
    $sort: { "totCoupon": -1 }
  }, {
    $limit: 1
  }
])
```

27. Per ogni metodo di acquisto trovare la media di soddisfazione delle vendite fatte con coupon e senza.

```
db.sales.aggregate([
  {
    $group: {
      _id: ["$purchaseMethod", "$couponUsed"],
      mediaSoddisfazione: { $avg: "$customer.satisfaction" }
    }
  }
])
```

28. Trovare il guadagno totale (sommatoria di price * quantity) per ogni tipo di item (campo "name").

```
db.sales.aggregate([
  {
    $unwind: "$items"
  }, {
    $group: {
      _id: "$items.name",
      totGuadagno: { $sum: { $multiply: ["$items.price",
"$items.quantity"] } }
    }
  }
])
```

29. Trovare il tag che compare più frequentemente tra tutte le vendite

```
db.sales.aggregate([
  {
    $unwind: "$items"
  }, {
    $unwind: "$items.tags"
  }, {
    $group: {
      _id: "$items.tags",
      counter: { $sum: 1 }
    }
  }, {
    $sort: { "counter": -1 }
  }, {
    $limit: 1
  }
])
```